

Assignment 9: Unsupervised Learning

Student Name: Mohammad Tamim Hamkar

Step 1: Import Libraries

```
1 # Import Libraries
2
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8 from sklearn.datasets import load_iris
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.cluster import KMeans, AgglomerativeClustering
11 from sklearn.metrics import silhouette_score
12 from sklearn.decomposition import PCA
13
14 from scipy.cluster.hierarchy import dendrogram, linkage
15
16 print("Libraries imported successfully.")
```

Libraries imported successfully.

Step 2: Load the Dataset

```
1 # Load the Iris Dataset
2
3 iris = load_iris()
4
5 df = pd.DataFrame(iris.data, columns=iris.feature_names)
6
7 df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step 3: Check and Clean the Data

```
1 # Check and Clean the Data
2
3 print("Dataset shape:", df.shape)
4 print("\nMissing values:", df.isnull().sum().sum())
5 print("Duplicate rows:", df.duplicated().sum())
```

```
6
7 df.info()
```

Dataset shape: (150, 4)

Missing values: 0

Duplicate rows: 1

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 150 entries, 0 to 149

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64

dtypes: float64(4)

memory usage: 4.8 KB

✓ Step 4: Remove Duplicate and Scale the Data

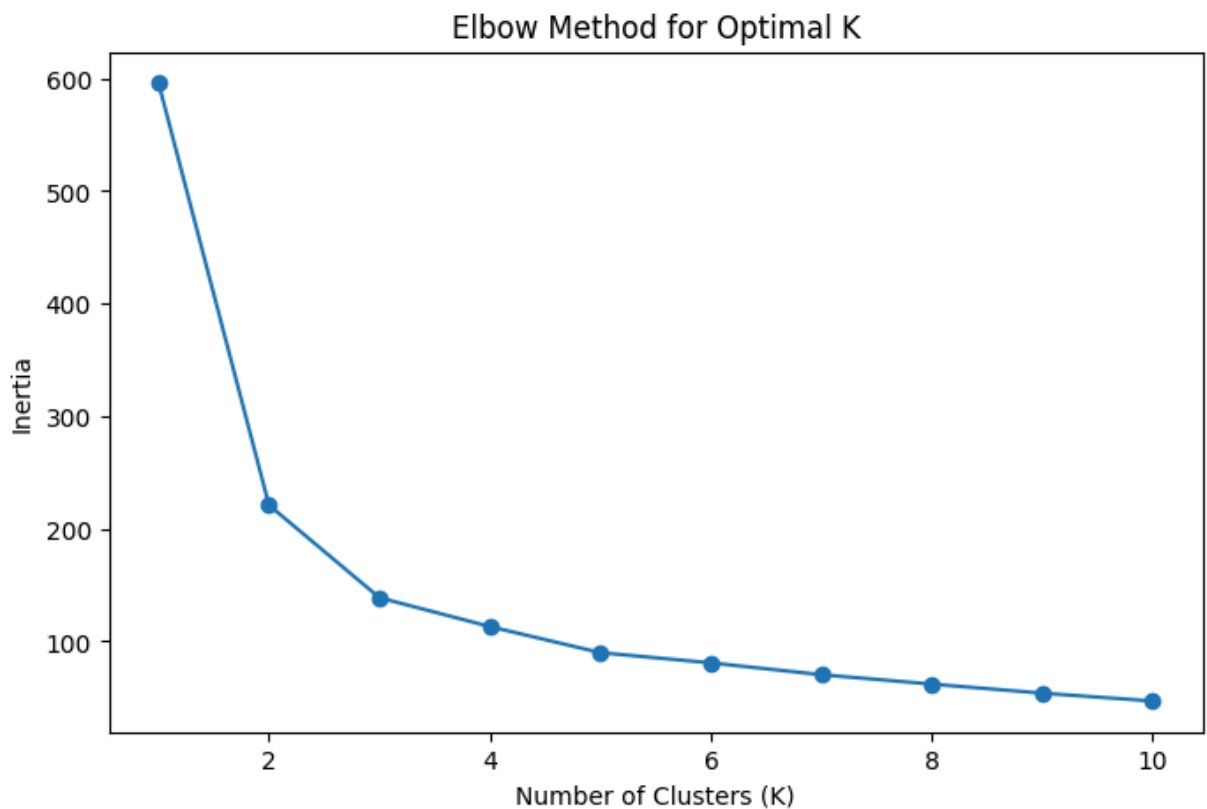
```
1 # Remove Duplicate and Scale the Data
2
3 df = df.drop_duplicates()
4
5 print("Dataset shape after removing duplicate:", df.shape)
6
7 scaler = StandardScaler()
8 scaled_data = scaler.fit_transform(df)
9
10 print("Feature scaling completed.")
```

Dataset shape after removing duplicate: (149, 4)

Feature scaling completed.

✓ Step 5: Find the Optimal Number of Clusters Using the Elbow Method

```
1 # Elbow Method
2
3 inertia = []
4
5 for k in range(1, 11):
6     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
7     kmeans.fit(scaled_data)
8     inertia.append(kmeans.inertia_)
9
10 plt.figure(figsize=(8, 5))
11 plt.plot(range(1, 11), inertia, marker='o')
12
13 plt.xlabel('Number of Clusters (K)')
14 plt.ylabel('Inertia')
15 plt.title('Elbow Method for Optimal K')
16 plt.show()
```



Step 6: Evaluate the Number of Clusters Using Silhouette Score

```
1 # Calculate Silhouette Scores
2
3 silhouette_scores = []
4
5 for k in range(2, 11):
6     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
7     labels = kmeans.fit_predict(scaled_data)
8
9     score = silhouette_score(scaled_data, labels)
10    silhouette_scores.append(score)
11
12    print(f"K = {k}: Silhouette Score = {score:.3f}")
```

```
K = 2: Silhouette Score = 0.581
K = 3: Silhouette Score = 0.461
K = 4: Silhouette Score = 0.388
K = 5: Silhouette Score = 0.346
K = 6: Silhouette Score = 0.344
K = 7: Silhouette Score = 0.327
K = 8: Silhouette Score = 0.331
K = 9: Silhouette Score = 0.349
K = 10: Silhouette Score = 0.350
```

Step 7: K-Means Clustering

```
1 # Apply K-Means Clustering
2
3 kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
4
5 kmeans_labels = kmeans.fit_predict(scaled_data)
6
7 df['KMeans_Cluster'] = kmeans_labels
```

```

8
9 print("K-Means Silhouette Score:",
10       silhouette_score(scaled_data, kmeans_labels))
11
12 print("\nNumber of samples in each cluster:")

```

K-Means Silhouette Score: 0.5810395072315084

Number of samples in each cluster:

KMeans_Cluster

1 99

0 50

Name: count, dtype: int64

✓ Step 8: Hierarchical Clustering

```

1 # Hierarchical Clustering
2
3 hierarchical = AgglomerativeClustering(n_clusters=2)
4
5 hierarchical_labels = hierarchical.fit_predict(scaled_data)
6
7 df['Hierarchical_Cluster'] = hierarchical_labels
8
9 hierarchical_score = silhouette_score(
10     scaled_data, hierarchical_labels
11 )
12
13 print("Hierarchical Clustering Silhouette Score:",
14       hierarchical_score)
15
16 print("\nNumber of samples in each cluster:")
17 print(df['Hierarchical_Cluster'].value_counts())

```

Hierarchical Clustering Silhouette Score: 0.5762985431787423

Number of samples in each cluster:

Hierarchical_Cluster

0 100

1 49

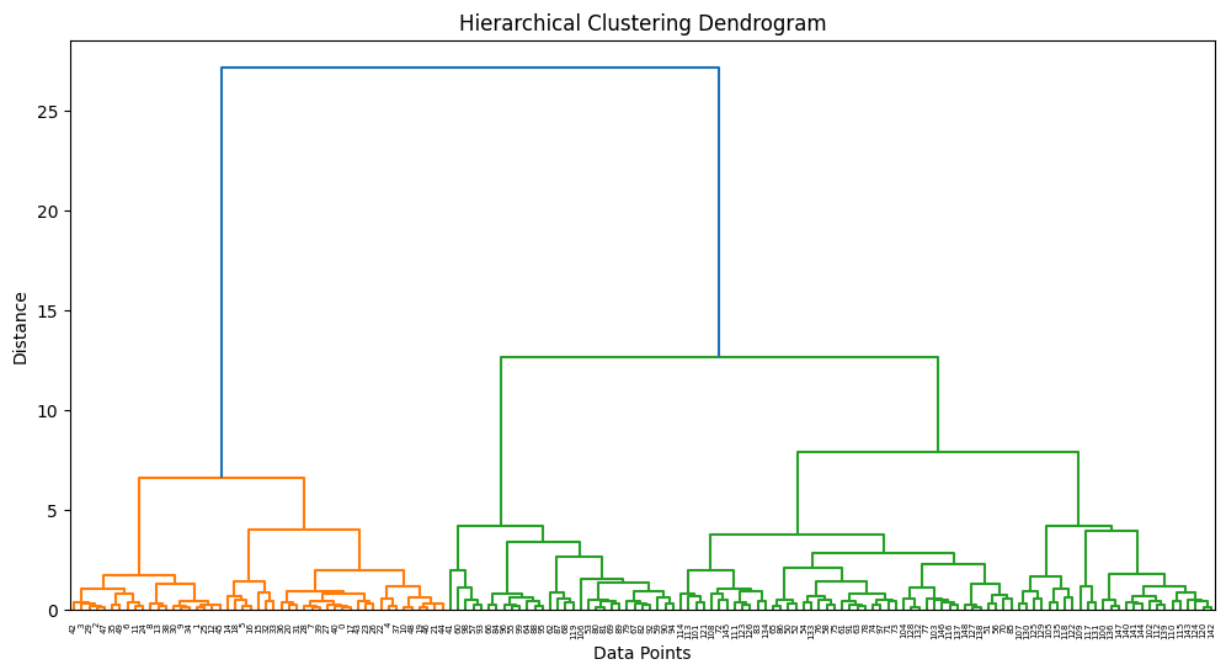
Name: count, dtype: int64

✓ Step 9: Hierarchical Clustering Dendrogram

```

1 # Create Dendrogram
2
3 linked = linkage(scaled_data, method='ward')
4
5 plt.figure(figsize=(12, 6))
6
7 dendrogram(linked)
8
9 plt.title('Hierarchical Clustering Dendrogram')
10 plt.xlabel('Data Points')
11 plt.ylabel('Distance')
12 plt.show()

```



✓ Step 10: PCA Dimensionality Reduction

```
1 # Apply PCA to Reduce the Data to 2 Dimensions
2
3 pca = PCA(n_components=2)
4
5 pca_data = pca.fit_transform(scaled_data)
6
7 print("Original dimensions:", scaled_data.shape)
8 print("PCA dimensions:", pca_data.shape)
9
10 print("\nExplained variance:")
11 print(pca.explained_variance_ratio_)
```

Original dimensions: (149, 4)
PCA dimensions: (149, 2)

Explained variance:
[0.72991022 0.22858172]

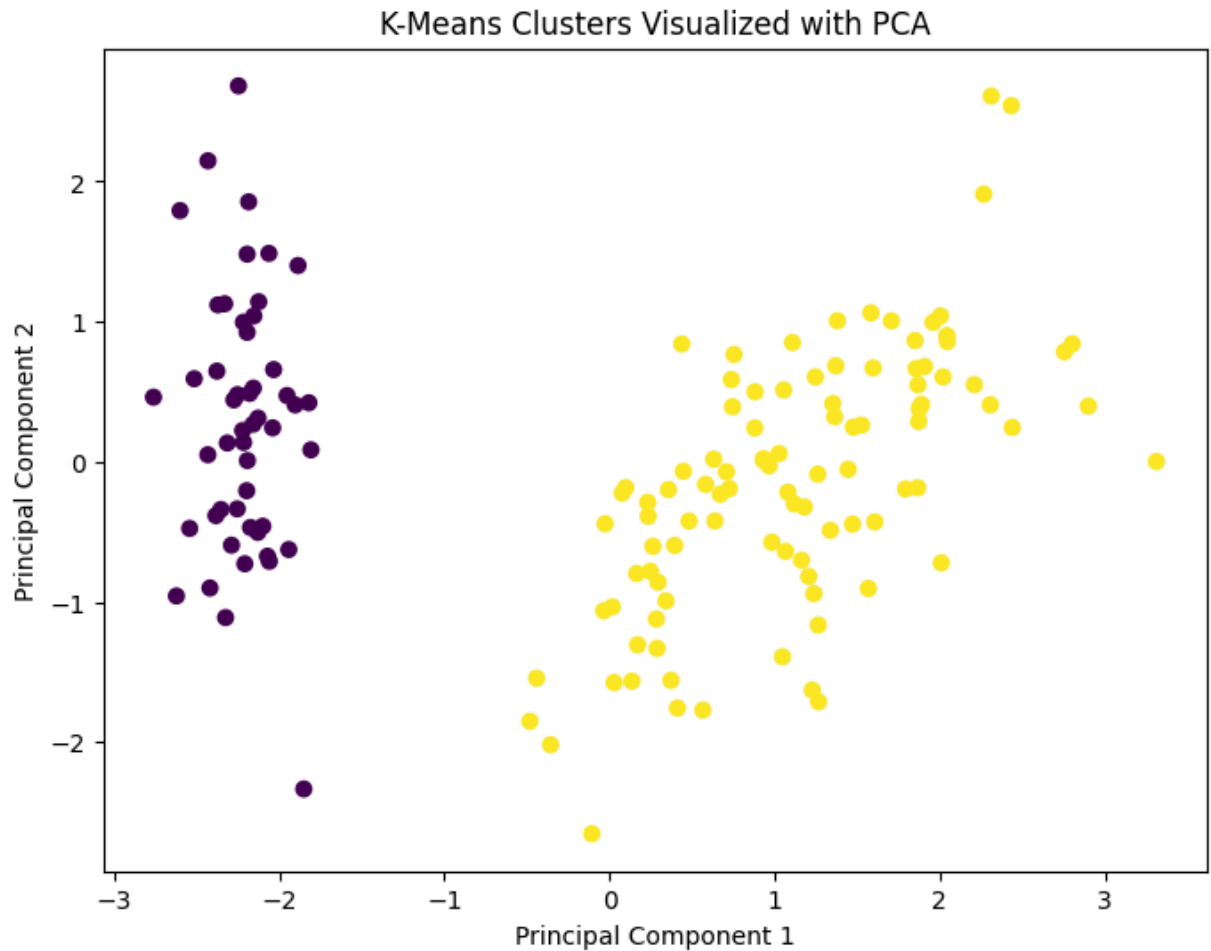
✓ Step 11: Visualize K-Means Clusters Using PCA

```
1 # Visualize K-Means Clusters Using PCA
2
3 plt.figure(figsize=(8, 6))
4
5 plt.scatter(
6     pca_data[:, 0],
7     pca_data[:, 1],
8     c=kmeans_labels,
9     cmap='viridis'
10 )
```

```

11
12 plt.xlabel('Principal Component 1')
13 plt.ylabel('Principal Component 2')
14 plt.title('K-Means Clusters Visualized with PCA')
15
16 plt.show()

```

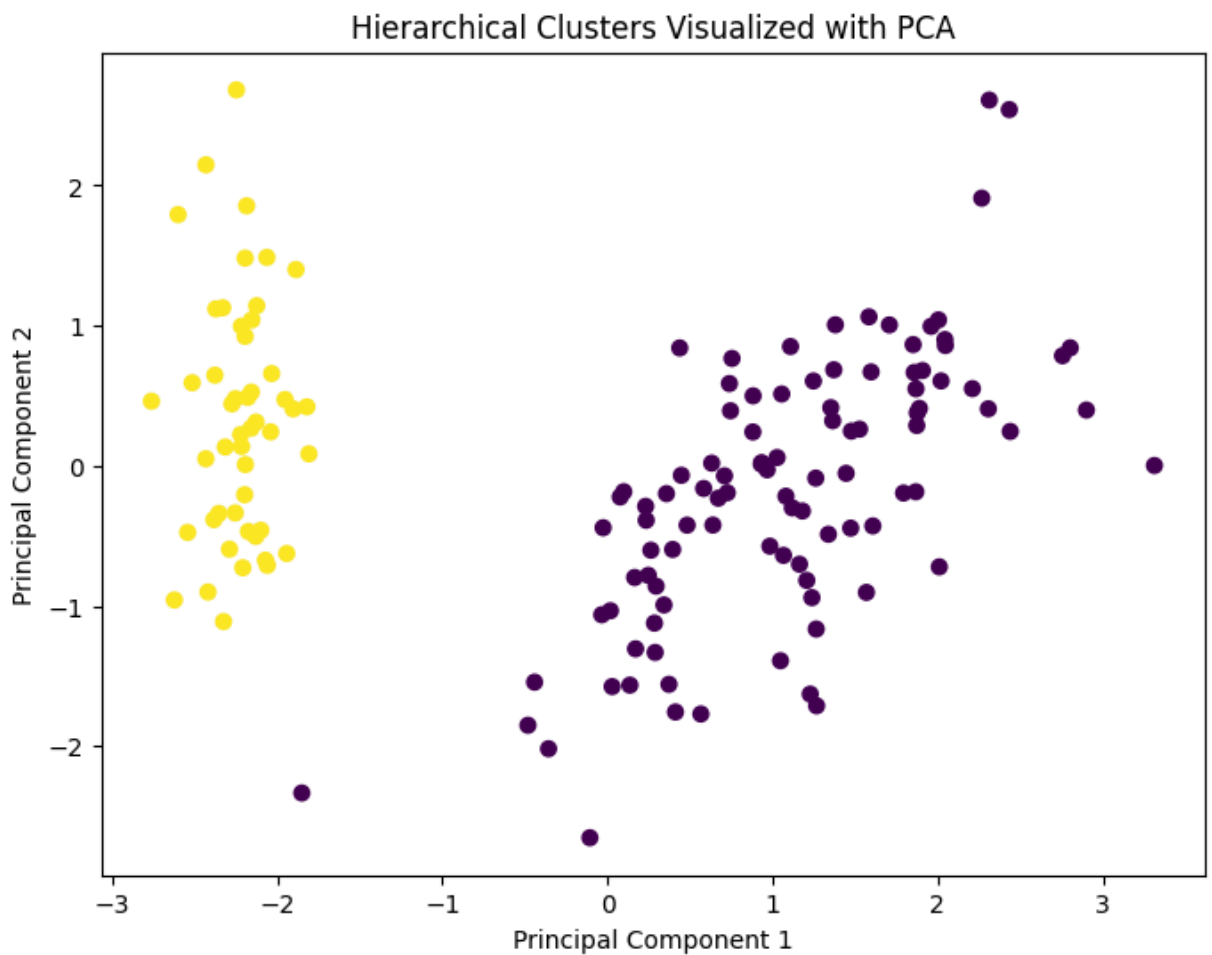


✓ Step 12: Visualize Hierarchical Clusters Using PCA

```

1 # Visualize Hierarchical Clusters Using PCA
2
3 plt.figure(figsize=(8, 6))
4
5 plt.scatter(
6     pca_data[:, 0],
7     pca_data[:, 1],
8     c=hierarchical_labels,
9     cmap='viridis'
10 )
11
12 plt.xlabel('Principal Component 1')
13 plt.ylabel('Principal Component 2')
14 plt.title('Hierarchical Clusters Visualized with PCA')
15
16 plt.show()

```



Step 13: Model Evaluation and Interpretation

Both K-Means and Hierarchical Clustering were able to separate the Iris data into meaningful groups.

K-Means achieved a Silhouette Score of 0.581, while Hierarchical Clustering achieved a score of 0.576. Since a higher Silhouette Score indicates better-defined clusters, K-Means performed slightly better.

The PCA visualizations also show that the data can be separated into two main groups. PCA reduced the original four features to two components while keeping about 95.85% of the information in the data.

Based on these results, I selected K-Means as the best-performing clustering model for this analysis.

Step 14: Deployment and Monitoring

If this K-Means model were deployed in a real-world application, it could be used to automatically group new data based on similar characteristics. New data would first need to go through the same preprocessing and scaling steps before being assigned to a cluster.

Some challenges could include handling large amounts of data, maintaining fast prediction times, and dealing with changes in the data over time.

The model should be monitored regularly to check whether the data distribution or cluster patterns are changing. The Silhouette Score can also be checked over time to see if the quality of the clusters is decreasing.

As new data becomes available, the model can be retrained and evaluated again. This would help keep the clustering results useful and reliable over time.

Step 15: Conclusion

In this assignment, I used unsupervised learning techniques to analyze the Iris dataset. I applied K-Means and Hierarchical Clustering and used the Silhouette Score to evaluate their performance.

K-Means achieved a Silhouette Score of 0.581, while Hierarchical Clustering achieved 0.576. Based on these results, K-Means performed slightly better.

I also used PCA to reduce the four original features to two components while keeping about 95.85% of the information. This made it easier to visualize the clusters.

Overall, this project showed how clustering and dimensionality reduction can be used to

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.